Université du Luxembourg

Introduction to Message Passing Interface (MPI)

Lesson 7

Matteo Barborini

University of Luxembourg - HPC Platform

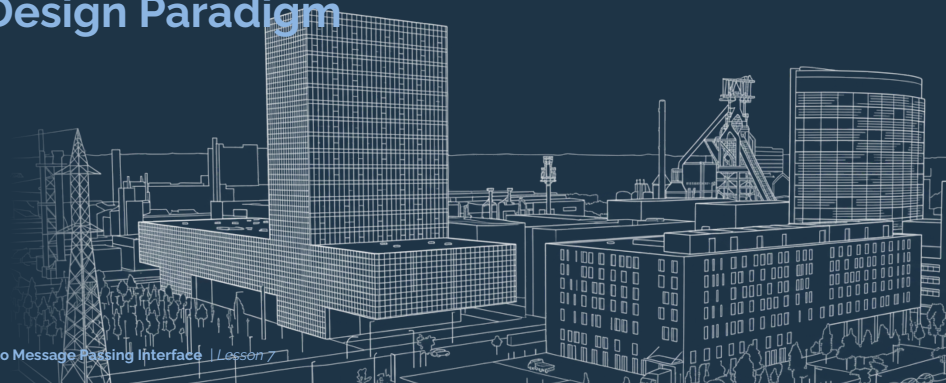
matteo.barborini@uni.lu

28th of October 2026



Université du Luxembourg

Foster's Design Paradigm



Foster's core concepts

Foster's Design Paradigm

In *"Designing and Building Parallel Programs"*[1], Ian Foster proposes a design model centered on **tasks** interacting through **communication channels**.

(In practice this methodology includes both shared memory and distributed memory paradigms)

Task: A program, its local memory, and its communication I/O ports.
A task can perform **four basic actions** in addition to reading and writing its local memory: send messages on its outports, receive messages on its inports, create new tasks, and terminate.

(In MPI these are generated with mpiexec/mpirun commands, where the number of tasks are controlled by the -np/-n commands. These create independent instances of my application)

A **parallel computation** consists of **one or more tasks execute concurrently** that can **vary in number** during program execution.

There is no unique mapping between tasks and physical processors; the mapping employed does not affect the semantics of a program.

Multiple tasks can be mapped to a single processor and a single task might be mapped to multiple processors.

[1] I. Foster, *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*, (Addison-Wesley Longman Publishing Co., 1995).

Foster's core concepts

Foster's Design Paradigm

Outport/inport pairs can be connected by **message queues called channels**.
Channels: Connects a task's 'in port' to another task's 'out port'.
Channels can be created and deleted, and references to channels (ports) can be included in messages, so connectivity can vary dynamically.

(In MPI these channels are initialized through the MPI libraries: the `void MPI_init();` command initializes the communication channels.)

Channels are buffered. (Remember that we discussed about this when discussing about the Send/Recv type of commands).
A send operation is asynchronous: it completes immediately. A receive operation is synchronous: it causes execution of the task to block until a message is available.
In MPI 'Standard' sending is **asynchronous**, while 'Standard' receiving is **synchronous** (receiving task blocks until expected message arrives).

(MPI is in fact push first)

Desired properties of the parallelization model

Foster's Design Paradigm

Other properties of this parallelization model: **performance**, **mapping independence**, **modularity**, and **determinism**.

Performance. The task and channel have a similarly direct mapping to the multicomputer. A task represents a piece of code that can be executed sequentially, on a single processor. If two tasks that share a channel are mapped to different processors, the channel connection is implemented as interprocessor communication; if they are mapped to the same processor, some more efficient mechanism can be used.

Mapping independence. Algorithms can be designed and implemented without concern for the number of processors on which they will execute.

Modularity. Modular program design, various components of a program are developed separately, as independent modules, and then combined to obtain a complete program.

Determinism. Parallel programming model that makes it easy to write deterministic programs is highly desirable. Deterministic programs tend to be easier to understand.

Université du Luxembourg

Examples of Scientific applications



Continuous partial differential equations (PDEs), for example the 2D (for representation limitations) Heat Equation

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u; \qquad \nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

can be discretized over a spatial mesh

$$\frac{u_{i,j}^{t+1} - u_{i,j}^{t}}{\Delta t} = \alpha \left(\frac{u_{i+1,j}^{t} - 2u_{i,j}^{t} + u_{i-1,j}^{t}}{\Delta x^2} + \frac{u_{i,j+1}^{t} - 2u_{i,j}^{t} + u_{i,j-1}^{t}}{\Delta y^2} \right)$$

thus the time evolution, assuming an identical spatial discretization ($\Delta x = \Delta y$) will be

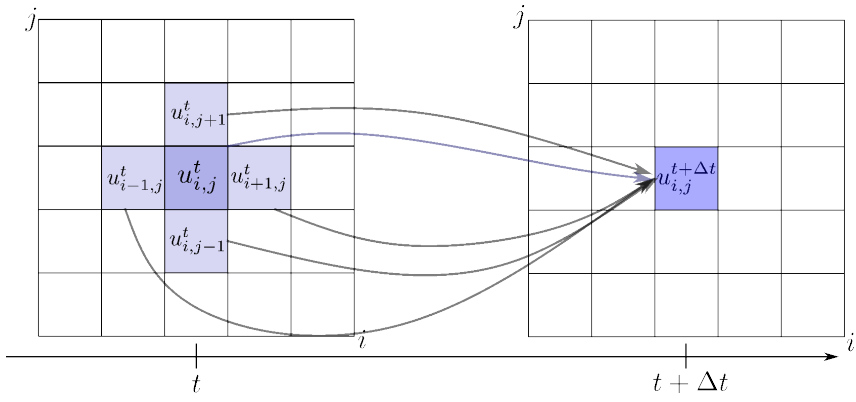
$$u_{i,j}^{t+1} = u_{i,j}^{t} + \frac{\alpha \Delta t}{\Delta x^2} \left(u_{i+1,j}^{t} + u_{i-1,j}^{t} + u_{i,j+1}^{t} + u_{i,j-1}^{t} - 4u_{i,j}^{t} \right)$$

Step $t+1$ cannot execute until all 4 spatial neighbors $(i \pm 1, j \pm 1)$ finish step t. Enforces strict temporal synchronization across iterations.

Finite Difference Methods

Examples of Scientific applications

$$u_{i,j}^{t+1} = u_{i,j}^{t} + \frac{\alpha \Delta t}{\Delta x^2} \left(u_{i+1,j}^{t} + u_{i-1,j}^{t} + u_{i,j+1}^{t} + u_{i,j-1}^{t} - 4u_{i,j}^{t} \right)$$



Pairwise interactions in Force Fields

Examples of Scientific applications

Total force acting on particle *i* due to all particles *j*:

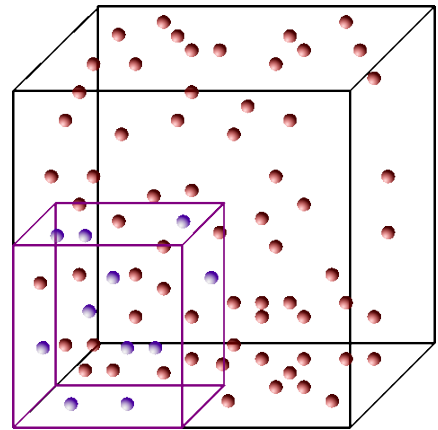
$$\mathbf{F}_i = \sum_{j=1(\neq i)}^{N} \mathbf{F}_{ij}(\mathbf{r}_i, \mathbf{r}_j)$$

Typical Force Field terms (e.g., Lennard-Jones):

$$V_{LJ}(r_{ij}) = 4\epsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^{6} \right].$$

Because of the symmetry $\mathbf{F}_{ij} = -\mathbf{F}_{ji}$ total interactions are $N(N-1)$.

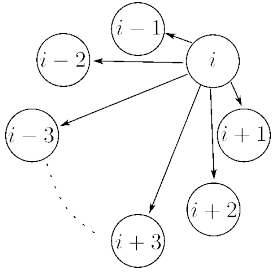
The parallelization of the algorithms require the splitting of the 3D space.



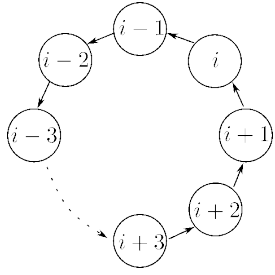
Pairwise interactions in Force Fields

Examples of Scientific applications

How to compute pairwise forces?



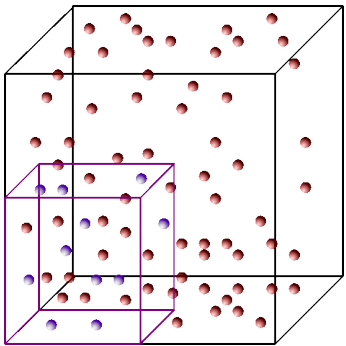
Global Dependency: Every particle interacts with every other particle ($\mathcal{O}(N^2)$ complexity).
Naive Memory Cost: Requires $\mathcal{O}(N^2)$ channels if all-to-all connectivity is used.
Scalability Barrier: All-to-all broadcast ($\mathcal{O}(N^2)$ communication volume) chokes network bandwidth.



Connect N processors in a **Unidirectional Ring** (N channels total).
Shift particle coordinates along the ring for $N-1$ steps.
Each step: Compute local pair interaction and accumulate force vectors.

(Add skip channels to return half-accumulated forces, halving communications to $N/2$)

Parallel Algorithms: Unidirectional Ring & Spatial Cutoffs



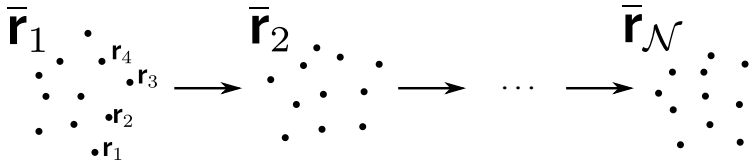
- Force fields drop off sharply: $\mathbf{F}_{ij} \approx 0$ for $r_{ij} > r_c$.
- **Spatial Grid (Cell-Linked Lists):** Divide box into cells of size $\geq r_c$.
- Processors only exchange particles with 26 adjacent 3D neighbor cells.
- Reduces complexity from $O(N^2)$ to $O(N)$.

Approach	Comm. Topology	Complexity
All-to-All	Complete Graph	$\mathcal{O}(N^2)$ Communications
Ring-Shift	1D Ring Topology	$\mathcal{O}(N)$ Communications
Spatial/Cell	3D Mesh (Halo)	$\mathcal{O}(N)$ Operations

Variational Monte Carlo (VMC)

Examples of Scientific applications

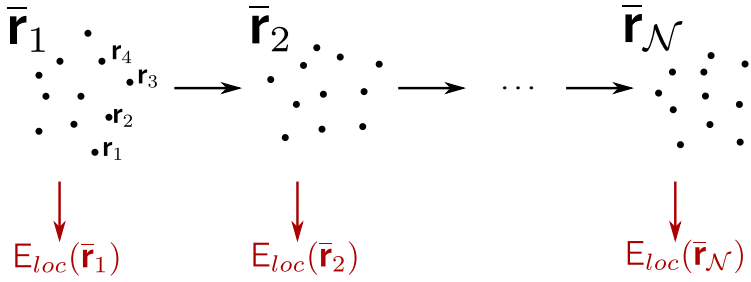
Random walk with $\mathcal{N}$ electronic configurations extracted with probability $p(\bar{\mathbf{r}}) \propto |\Psi_T(\bar{\mathbf{r}})|^2$



Variational Monte Carlo (VMC)

Examples of Scientific applications

Random walk with $\mathcal{N}$ electronic configurations extracted with probability $p(\bar{\mathbf{r}}) \propto |\Psi_T(\bar{\mathbf{r}})|^2$

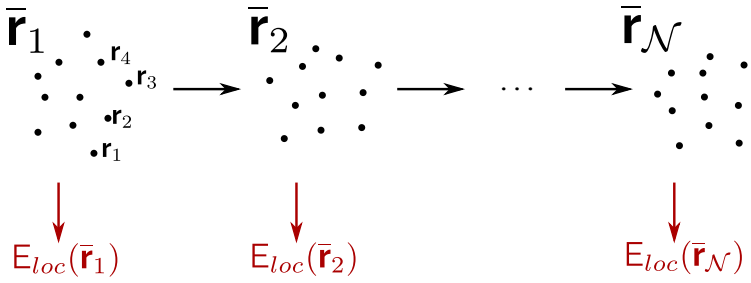


For each configuration we compute $E_{loc}(\bar{\mathbf{r}}) = \frac{\hat{H}_e \Psi_T(\bar{\mathbf{r}})}{\Psi_T(\bar{\mathbf{r}})}$

Variational Monte Carlo (VMC)

Examples of Scientific applications

Random walk with $\mathcal{N}$ electronic configurations extracted with probability $p(\bar{\mathbf{r}}) \propto |\Psi_T(\bar{\mathbf{r}})|^2$



For each configuration we compute $E_{loc}(\bar{\mathbf{r}}) = \frac{\hat{H}_e \Psi_T(\bar{\mathbf{r}})}{\Psi_T(\bar{\mathbf{r}})}$

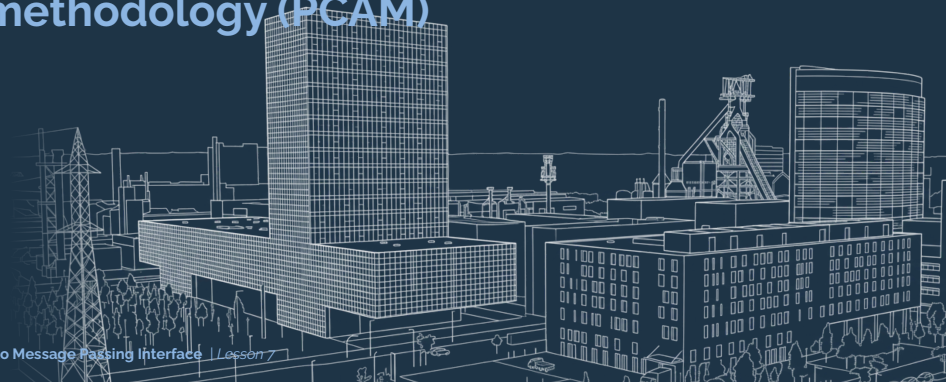
The estimation of the energy functional corresponds to the mean value of the Local energies:

$$E[\Psi_T] \approx \langle E_{loc} \rangle_{\mathcal{N}} \pm \sqrt{\frac{\langle E_{loc}^2 \rangle_{\mathcal{N}} - \langle E_{loc} \rangle_{\mathcal{N}}^2}{\mathcal{N}}}$$

Error decreases has $1/\sqrt{N}$ and the variance is zero for the exact wave function

Université du Luxembourg

Foster's methodology (PCAM)



Definition on PCAM

Foster's methodology (PCAM)

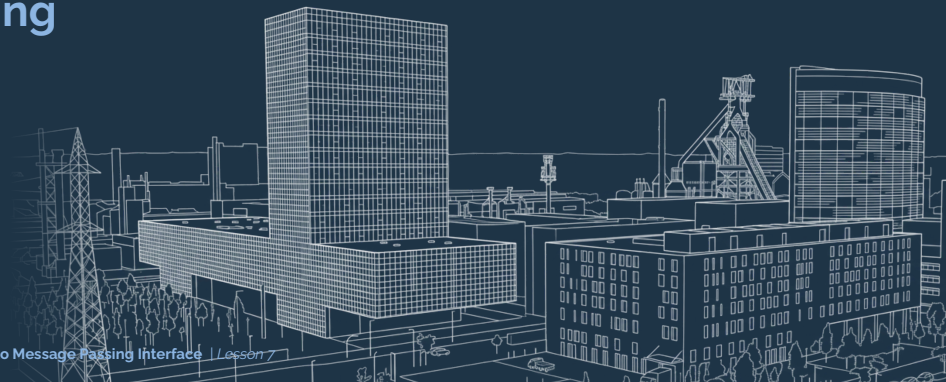
Ian Foster's methodology[2] is a systematic approach to designing parallel programs. It consists of four main steps:

1. **Partitioning (P):** Divide the problem into smaller tasks that can be executed concurrently. This involves identifying independent tasks and determining how to distribute the workload among available processors.
2. **Communication (C):** Determine how the tasks will communicate and share data. This step involves designing the communication patterns and protocols to ensure that tasks can exchange information efficiently.
3. **Agglomeration (or aggregation) (A):** Combine tasks to reduce communication overhead. The tasks that need to be done sequentially should be grouped on one process/thread.
4. **Mapping (M):** Assign tasks to specific processors/threads in the parallel system. This step involves determining the optimal placement of tasks to minimize communication costs and maximize performance.

[2] I. Foster, *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*, (Addison-Wesley Longman Publishing Co., 1995).

Université du Luxembourg

Partitioning



Partitioning

Partitioning

Focuses on defining a large number of small tasks (**fine-grained decomposition**).

A good partition divides into small pieces both the computation associated with a problem and the data on which this computation operates, discover as much parallelism as possible.

Domain Decomposition
First partition data, then associate computation operations with the data.

Functional Decomposition
First decompose computation into disjoint tasks, then examine data requirements.

Domain and functional decompositions are complementary techniques, and the idea is to avoid replicating computation and data as much as possible in this initial phase.

(Clearly the type of decomposition also depends on the type of operation that has to be parallelized)

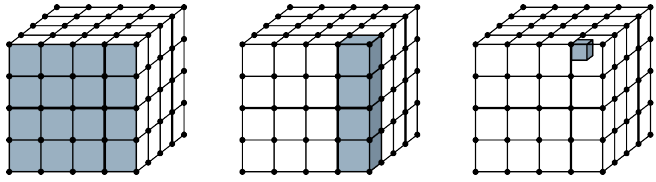
Domain Decomposition

Partitioning

Focuses on the underlining data structure.

1. **Partition Data:** Divide data into small pieces of approximately equal size.
2. **Partition Computation:** Associate operations with the data on which they operate.

(First decompose the largest data structures or the most frequently accessed ones.)



1D Decomposition: Split data into a 1-D array of 2-D slices (coarse granularity).

2D Decomposition: Split data into a 2-D array of 1-D columns.

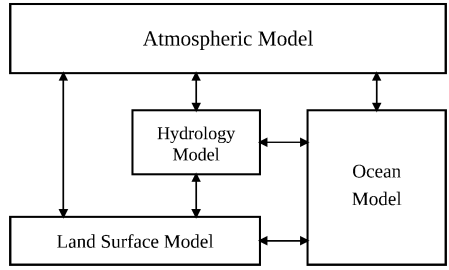
3D Decomposition: Split data into individual 3-D elements (fine granularity).

Functional Decomposition

Partitioning

Focuses primarily on the computation to be performed rather than the underlying data structures.

1. **Partition Computation:** Divide computation into disjoint tasks.
2. **Partition Data:** Examine data requirements for each task:
   - If data requirements are **disjoint**, the partition is complete.
   - If requirements **overlap significantly**, substantial communication is required to avoid data replication (a sign that Domain Decomposition may be better).



Foster's example is that of atmospheric models, but in plasma physics we have the particle-in-cell (PIC) methods.

(Reveals problem structure/optimizations not obvious from data alone, and provides a powerful program structuring technique.)

Partitioning Design Checklist

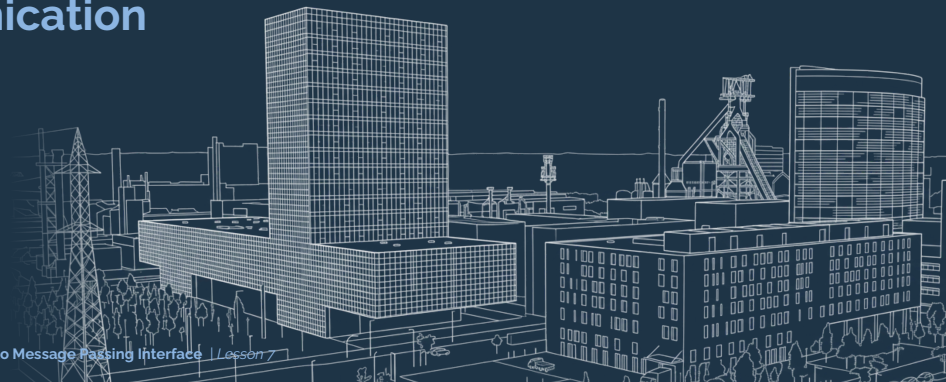
Partitioning

Questions to evaluate before finishing the partitioning stage:

1. Does your partition define at least an order of magnitude more tasks than processors in the target hardware?
2. Does your partition avoid redundant computation and storage requirements?
3. Are tasks of comparable size?
4. Does the number of tasks scale with problem size?
5. Have you identified several alternative partitions?

Université du Luxembourg

Communication



Communication Overview

5 Communication

The information flow between concurrent tasks defined in the partitioning phase is specified in the communication phase of a design, in which **communication channels** are established.

The definition of a channel involves an intellectual cost (MPI parallelism) and the sending of a message involves a physical cost (MPI distribution functions).

We seek to optimize performance by distributing communication operations over many tasks and by organizing communication operations in a way that permits concurrent execution.

In **domain decomposition** communication requirements can be difficult to determine

- First the data structures are partitioned into disjoint subsets and then associate to the local data those operations that operate solely on that local data.
- Often there are operations that require local data from several tasks, increasing communication (dgemv,dgemm, discretized PES ecc.).

In **functional decomposition** communication requirements usually correspond to the data flow between tasks. Within heterogeneous architectures it might be useful to parallelize different tasks on different architectures.

Patterns of Communication

5 Communication

Foster categorizes communication along four loosely orthogonal axes:

Local vs. Global

Local: Each task communicates with a small set of neighbors.

Global: Requires each task to communicate with many tasks.

Static vs. Dynamic

Static: Communication partners do not change over time.

Dynamic: Partners are determined at runtime based on data.

Structured vs. Unstructured

Structured: Tasks and neighbors form regular topologies (trees, grids).

Unstructured: Communication graph forms an arbitrary network.

Synchronous vs. Asynchronous

Synchronous: Producer/consumer cooperate directly.

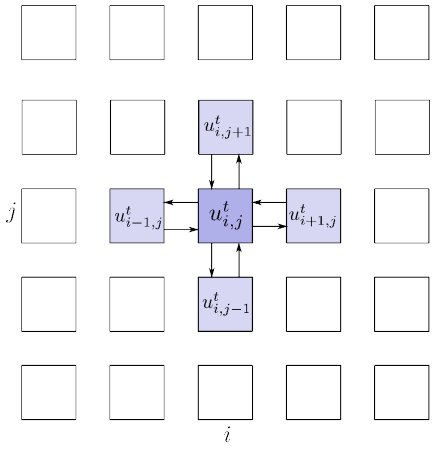
Asynchronous: Consumer requests data independently without active producer participation.

Local Communication Example: Jacobi Iteration

For parallel implementation of Jacobi iteration for finite differences:

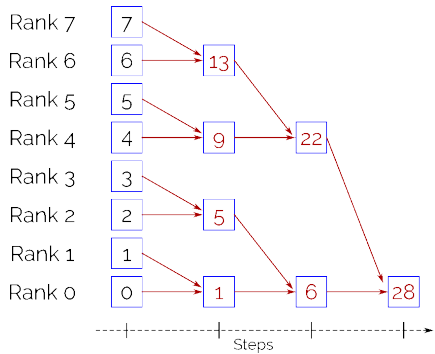
$$u_{i,j}^{t+1} = u_{i,j}^{t} + \frac{\alpha \Delta t}{\Delta x^2} \left(u_{i+1,j}^{t} + u_{i-1,j}^{t} + \right.$$
$$\left. + u_{i,j+1}^{t} + u_{i,j-1}^{t} - 4u_{i,j}^{t} \right) \quad (1)$$

If we assume one task per grid points we have the communication pattern on the right



Consider parallel reduction summing N numbers ($S = \sum X_i$):

- **Centralized Manager:** Single manager receives N values sequential $O(N)$ time (poor parallel performance).
- **Pipeline Array:** Pass partial sum $S_i = X_i + S_{i-1}$ along a 1D chain of tasks. $O(N)$ latency, but enables pipelining for multiple sums.
- **Divide and Conquer:** Binary tree reduction reduces latency to $O(\log N)$ steps.



Communication Checklist

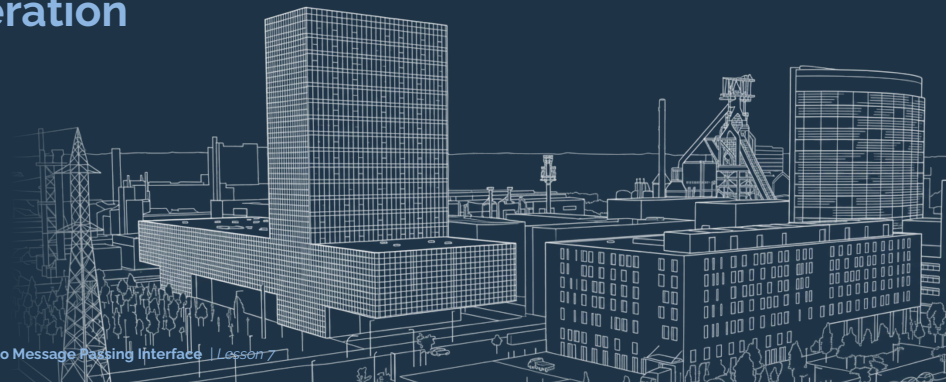
5 Communication

Questions to evaluate before finishing the communication step:

1. Do all tasks perform about the same number of communication operations?
2. Does each task communicate with only a small number of neighbors?
3. Are communication operations able to proceed concurrently?
4. Is computation associated with different tasks able to proceed concurrently?

Université du Luxembourg

Agglomeration



Agglomeration Overview

6 Agglomeration

In this phase we revisit decisions made in the partitioning and communication phases:

- Combine fine-grained tasks into larger tasks to reduce communication overhead and improve efficiency.
- Evaluate whether data or computation replication is beneficial, to accelerate the computation.

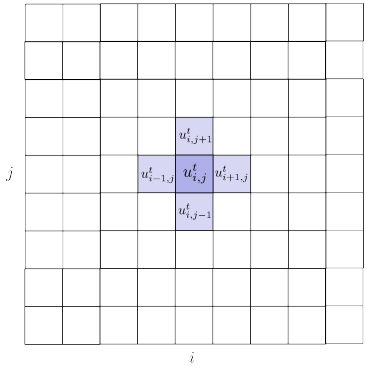
We may choose during the agglomeration phase to reduce the number of tasks to exactly one per processor.

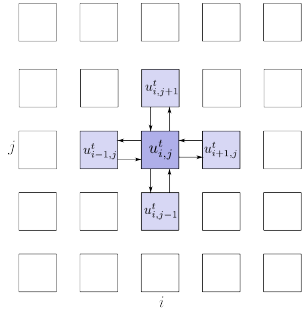
Three sometimes-conflicting goals guide decisions concerning agglomeration and replication:

1. increasing granularity: Reduce communication costs;
2. preserve flexibility and scalability for mapping onto variable processor counts;
3. reduce software engineering costs.

Surface-to-Volume Effects

6 Agglomeration



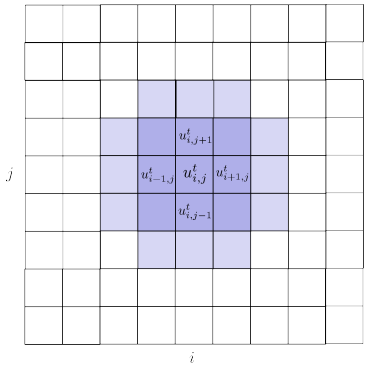


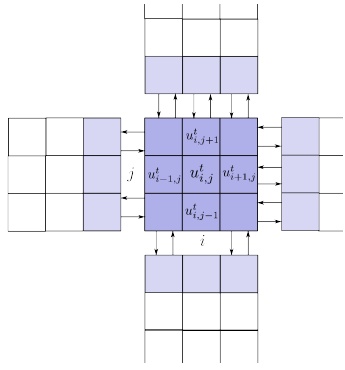
For a full connected grid I would need $4 \times N^2$ communications.

Agglomerating tasks improves efficiency due to the **surface-to-volume ratio**!

Surface-to-Volume Effects

6 Agglomeration





If we agglomerate the data in blocks of 3×3 elements, the communications between blocks would be of 4×3 data structures repeated for $N^2/3^2$ blocks.

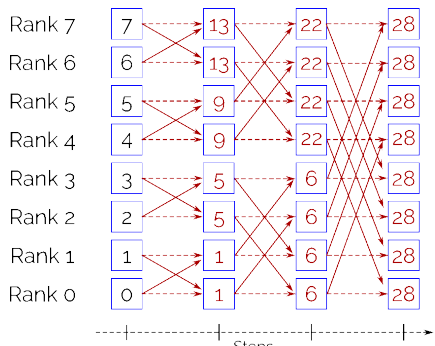
Computation scales with subdomain **volume** (e.g., N^2), while communication scales with subdomain **surface** (e.g., N). Agglomerating in all dimensions reduces communication ratio.

Replicating Computation

6 Agglomeration

It is sometimes faster to recompute data locally than to communicate it from another task.

- **Tree Sum + Broadcast:** $O(\log N)$ sum followed by $O(\log N)$ broadcast = $2 \log N$ communication steps.
- **Ring Summation:** All N tasks pass partial sums around a ring concurrently. Requires $N - 1$ steps with zero broadcast overhead, at the cost of $(N - 1)^2$ redundant additions.
- **Butterfly Pattern:** Modifies reduction so every task ends up with a copy of the total sum after $\log N$ steps ($O(N \log N)$ total operations).



Additional Considerations

6 Agglomeration

Avoiding communication: Agglomeration is almost always beneficial if analysis of communication requirements reveals that a set of tasks cannot execute concurrently.

Preserving flexibility:

- It is important when agglomerating to avoid making design decisions that limit unnecessarily an algorithm's scalability. Don't assume during the design that the number of processors will always be limited to the currently available number.
- Good parallel algorithms are designed to be resilient to changes in processor count.
- It can be advantageous to map several tasks to a processor. Then, a blocked task need not result in a processor becoming idle, since another task may be able to execute in its place.

Reducing software engineering costs

- An additional concern, which can be particularly important when parallelizing existing sequential codes, is the relative development costs associated with different partitioning strategies.
- The most useful strategies may be those that avoid extensive code changes, can make use of existing code, or are required by other constraints (e.g. interface to other software products)

Agglomeration Checklist

6 Agglomeration

1. Has agglomeration reduced communication costs by increasing locality? If not, examine your algorithm to determine whether this could be achieved using an alternative agglomeration strategy.

2. If agglomeration replicates computation, have you verified that the benefits of this replication outweigh its costs, for a range of problem sizes and processor counts?

3. If agglomeration replicates data, have you verified that this does not compromise the scalability of your algorithm by restricting the range of problem sizes or processor counts that it can address?

4. Has agglomeration yielded load-balanced tasks? The larger the tasks created by agglomeration, the more important it is that they have similar costs. If we have created just one task per processor, then these tasks should have nearly identical costs.

5. Does the number of tasks still scale with problem size? If not, then your algorithm is no longer able to solve larger problems on larger parallel computers.

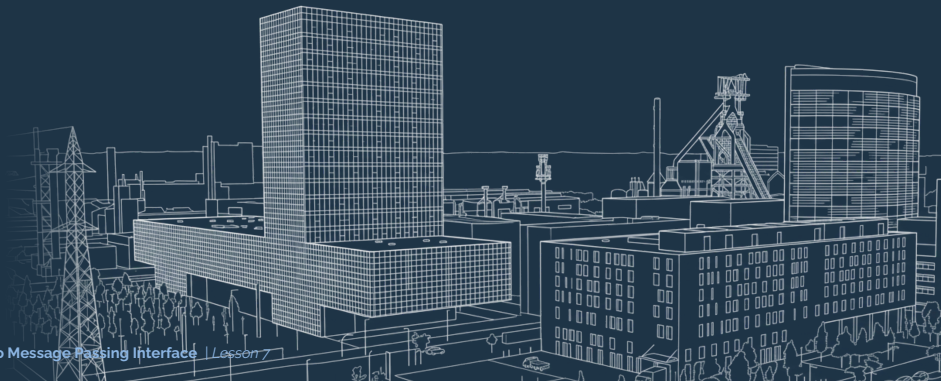
6. If agglomeration eliminated opportunities for concurrent execution, have you verified that there is sufficient concurrency for current and future target computers? An algorithm with insufficient concurrency may still be the most efficient, if other algorithms have excessive communication costs; performance models can be used to quantify these tradeoffs.

7. Can the number of tasks be reduced still further, without introducing load imbalances, increasing software engineering costs, or reducing scalability? Other things being equal, algorithms that create fewer larger-grained tasks are often simpler and more efficient than those that create many fine-grained tasks.

8. If you are parallelizing an existing sequential program, have you considered the cost of the modifications required to the sequential code? If these costs are high, consider alternative agglomeration strategies that increase opportunities for code reuse. If the resulting algorithms are less efficient, use performance modeling techniques to estimate cost tradeoffs.

Université du Luxembourg

Mapping



Mapping Overview

7 Mapping

In the **mapping** step we assign agglomerated tasks to physical processors on the target architecture.

The mapping step arises in distributed memory parallelization (MPI) it is usually not present in shared-memory parallelization, or when automatic parallelization of tasks is handled by the hardware.

This step has two main objectives:

1. Maximize concurrency by placing concurrent tasks on different processors.
2. Maximize locality by placing frequently communicating tasks on the same processor.

The mapping problem is known to be NP-complete (Nondeterministic Polynomial time), meaning that no computationally tractable (polynomial-time) algorithm can exist for evaluating these tradeoffs in the general case.

However, considerable knowledge has been gained on specialized strategies and heuristics and the classes of problems for which they are effective.

When domain decomposition is used there is often a fixed number of equal-sized tasks and structured local and global communication.

Load Balancing Strategies

7 Mapping

If functional decomposition is used we can use task-scheduling algorithms which allocate tasks to processors that are idle or that are likely to become idle.

Recursive Bisection Recursively cuts domain into subdomains of equal computational cost while minimizing boundary cross-edges.

- **Coordinate Bisection:** Cuts along spatial coordinates.
- **Unbalanced Bisection:** Adjusts sub-grid aspect ratios to reduce communication boundaries.
- **Graph Bisection:** Uses connectivity graph topology.

Local Algorithms periodically, each processor compares its computational load with that of its *neighbors* and transfers computation if the difference in load exceeds some threshold

Probabilistic Methods randomly selected processors. If the number of tasks is large, we can expect that each processor will be allocated about the same amount of computation.

Cyclic (or scattered) Mappings each of P processors is allocated every P*th* task according to some enumeration of the tasks.

Task-Scheduling Algorithms

7 Mapping

Task-scheduling algorithms can be used when a functional decomposition yields many tasks, each with weak locality requirements.

Manager/Worker A central manager task is given responsibility for problem allocation. Each worker repeatedly requests and executes a problem from the manager. Workers can also send new tasks to the manager for allocation to other workers.

Hierarchical divides workers into disjoint sets, each with a submanager. Workers request tasks from submanagers, which themselves communicate periodically with the manager and with other submanagers to balance load between the sets of processors for which they are responsible.

Decentralized there is no central manager. Each processor maintains a local task pool and idle workers request problems from other processors. .

Task-scheduling algorithms require a mechanism for determining when a search is complete; otherwise, idle workers will never stop requesting work from other workers.
This **termination detection operation** is straightforward in centralized schemes, because the manager can easily determine when all workers are idle. It is more difficult in decentralized algorithms

Mapping Design Checklist

7 Mapping

Questions to consider before finalizing mapping:

1. Have you considered both Single Program Multiple Data (SPMD) and dynamic task creation/deletion approaches?

2. If using a centralized manager, have you verified it won't become a bottleneck?

3. If using dynamic load balancing, have you evaluated algorithm overhead versus load imbalance costs?

4. If using probabilistic or cyclic mapping, are there enough tasks to ensure reasonable load distribution?